# Exercice 3 : Gestion d'un personnel universitaire

## Corrigé – Programmation Orientée Objet en C++

| | |
|---|---|
| Nom et prénoms : | ..... |
| Filière / Classe : | ..... |
| Enseignant : | ..... |
| Date : | 13 novembre 2026 |

## 1) Classe mère et classes dérivées

Tous les employés ont un matricule, un nom, un prénom et un salaire de base : ces attributs communs sont placés dans la **classe mère**. Ce qui diffère (indemnités, primes) est placé dans les **classes dérivées**.

| Classe | Rôle | Attributs propres |
|---|---|---|
| **Employe** | Classe mère **abstraite** | matricule, nom, prenom, salaireBase |
| **Enseignant** | Classe dérivée | indemniteEnseignement, indemniteRecherche |
| **AgentAdministratif** | Classe dérivée | primeRendement, primeResponsabilite |
| **Technicien** | Classe dérivée | primeTechnicite, primeRisque |

*Employe* est abstraite : il n'existe pas d'« employé » sans catégorie, et le calcul du salaire dépend de la catégorie.

## 2) Diagramme de classes UML

*Employe* est représentée en italique (classe abstraite) ; les classes dérivées ajoutent leurs attributs et redéfinissent `calculerSalaire()` et `afficher()`.

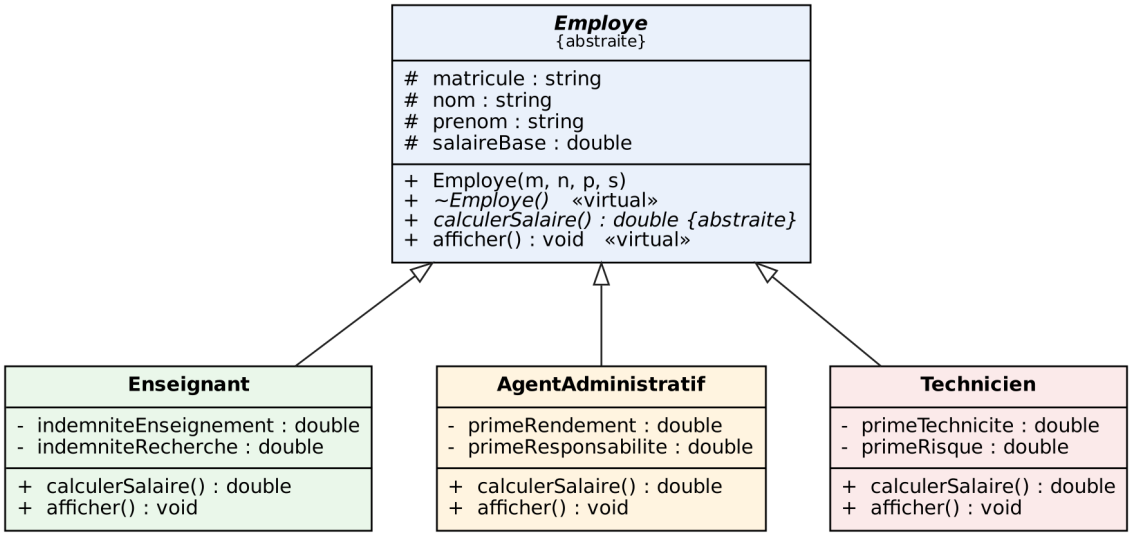



*Figure 1 – Diagramme de classes (# protégé, - privé, + public)*

## 3) Relations d'héritage

L'héritage se représente par une **flèche à pointe triangulaire creuse** allant de la classe dérivée vers la classe mère. Chaque relation se lit « *est un* » : un enseignant est un employé, un agent administratif est un employé, un technicien est un employé.

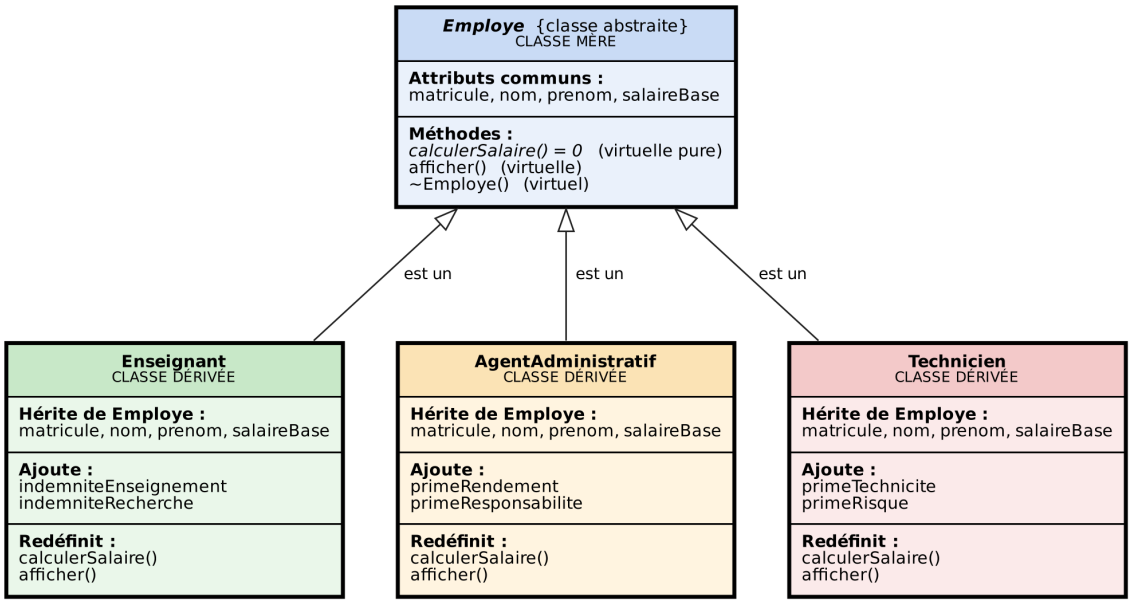



*Figure 2 – Hiérarchie d'héritage*

En C++ : `class Enseignant : public Employe`, `class AgentAdministratif : public Employe`, `class Technicien : public Employe`. Le détail des diagrammes (classes, héritage, objets, séquence) est dans *Exercice3_Modelisation_UML.pdf*.

## 4) Implémentation en C++

Fichier `personnel.cpp` (compilation : `g++ -std=c++11 -Wall -Wextra personnel.cpp -o personnel`).

```
// ============================================================
//  Exercice 3 - Gestion d'un personnel universitaire
// ============================================================
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

// ------------- Classe mere abstraite : Employe -------------
class Employe {
protected:
    string matricule, nom, prenom;
    double salaireBase;
public:
    Employe(string m, string n, string p, double s)
        : matricule(m), nom(n), prenom(p), salaireBase(s) {}

    virtual ~Employe() {}                          // destructeur virtuel

    virtual double calculerSalaire() const = 0;    // methode virtuelle pure

    virtual void afficher() const {                // methode virtuelle
        cout << "Matricule       : " << matricule << endl;
        cout << "Nom et prenom   : " << nom << " " << prenom << endl;
        cout << "Salaire de base : " << salaireBase << " FCFA" << endl;
    }
};

// ------------------ Classe derivee : Enseignant ------------------
class Enseignant : public Employe {
    double indemniteEnseignement, indemniteRecherche;
```

```
public:
    Enseignant(string m, string n, string p, double s, double ie, double ir)
        : Employe(m, n, p, s), indemniteEnseignement(ie), indemniteRecherche(ir) {}

    double calculerSalaire() const override {
        return salaireBase + indemniteEnseignement + indemniteRecherche;
    }
    void afficher() const override {
        cout << "--- Enseignant ---" << endl;
        Employe::afficher();
        cout << "Indemnite d'enseignement : " << indemniteEnseignement << " FCFA" << endl;
        cout << "Indemnite de recherche   : " << indemniteRecherche << " FCFA" << endl;
        cout << "Salaire total   : " << calculerSalaire() << " FCFA" << endl;
    }
};

// --------------- Classe derivee : AgentAdministratif ---------------
class AgentAdministratif : public Employe {
    double primeRendement, primeResponsabilite;
public:
    AgentAdministratif(string m, string n, string p, double s, double pr, double presp)
        : Employe(m, n, p, s), primeRendement(pr), primeResponsabilite(presp) {}

    double calculerSalaire() const override {
        return salaireBase + primeRendement + primeResponsabilite;
    }
    void afficher() const override {
        cout << "--- Agent administratif ---" << endl;
        Employe::afficher();
        cout << "Prime de rendement       : " << primeRendement << " FCFA" << endl;
        cout << "Prime de responsabilite  : " << primeResponsabilite << " FCFA" << endl;
        cout << "Salaire total   : " << calculerSalaire() << " FCFA" << endl;
    }
};

// ------------------- Classe derivee : Technicien -------------------
class Technicien : public Employe {
    double primeTechnicite, primeRisque;
public:
    Technicien(string m, string n, string p, double s, double pt, double pr)
        : Employe(m, n, p, s), primeTechnicite(pt), primeRisque(pr) {}

    double calculerSalaire() const override {
        return salaireBase + primeTechnicite + primeRisque;
    }
    void afficher() const override {
        cout << "--- Technicien ---" << endl;
        Employe::afficher();
        cout << "Prime de technicite      : " << primeTechnicite << " FCFA" << endl;
        cout << "Prime de risque          : " << primeRisque << " FCFA" << endl;
        cout << "Salaire total   : " << calculerSalaire() << " FCFA" << endl;
    }
};

// ------------------------ Programme principal ------------------------
int main() {
    cout << fixed << setprecision(0);               // affichage des montants sans notation scientifique

    // Employe e("M0", "X", "Y", 1000);   // ERREUR : Employe est abstraite

    // Collection contenant plusieurs categories d'employes
    vector<Employe*> personnel;
    personnel.push_back(new Enseignant("ENS001", "Adjovi", "Koffi", 450000, 120000, 80000));
    personnel.push_back(new Enseignant("ENS002", "Dossou", "Mariam", 500000, 150000, 100000));
    personnel.push_back(new AgentAdministratif("ADM001", "Hounkpatin", "Sylvie", 250000, 40000, 30000));
    personnel.push_back(new Technicien("TEC001", "Gbaguidi", "Rodrigue", 220000, 35000, 25000));

    double masseSalariale = 0;
    for (Employe* e : personnel) {
        e->afficher();                              // liaison dynamique
        masseSalariale += e->calculerSalaire();     // liaison dynamique
        cout << "-----------------------------------" << endl;
    }
    cout << "Masse salariale : " << masseSalariale << " FCFA" << endl;

    for (Employe* e : personnel) delete e;          // liberation de la memoire
    return 0;
}
```

## 5) Méthodes virtuelles et calcul du salaire

- Dans *Employe* : `virtual double calculerSalaire() const = 0;` (méthode virtuelle pure, sans corps).
- Chaque classe dérivée la **redéfinit** avec `override` :

| Classe | Salaire calculé |
|---|---|
| Enseignant | salaireBase + indemniteEnseignement + indemniteRecherche |
| AgentAdministratif | salaireBase + primeRendement + primeResponsabilite |
| Technicien | salaireBase + primeTechnicite + primeRisque |

- Le destructeur `virtual ~Employe()` garantit que `delete` sur un `Employe*` détruit correctement l'objet réel.

## 6) Affichage des informations

La méthode `afficher()` est virtuelle. Dans *Employe*, elle affiche les informations communes ; chaque classe dérivée la redéfinit, appelle `Employe::afficher()` puis affiche ses primes et le salaire total.

## 7) Collection d'employés de plusieurs catégories

On utilise un `vector<Employe*>` : un pointeur sur la classe mère peut désigner n'importe quelle classe dérivée. Dans la boucle, `e->afficher()` et `e->calculerSalaire()` exécutent la version de la classe **réelle** de l'objet : c'est la **liaison dynamique** (polymorphisme).

```
vector<Employe*> personnel;
personnel.push_back(new Enseignant("ENS001", "Adjovi", "Koffi", 450000, 120000, 80000));
personnel.push_back(new Technicien("TEC001", "Gbaguidi", "Rodrigue", 220000, 35000, 25000));
for (Employe* e : personnel)
    e->afficher();          // liaison dynamique
```

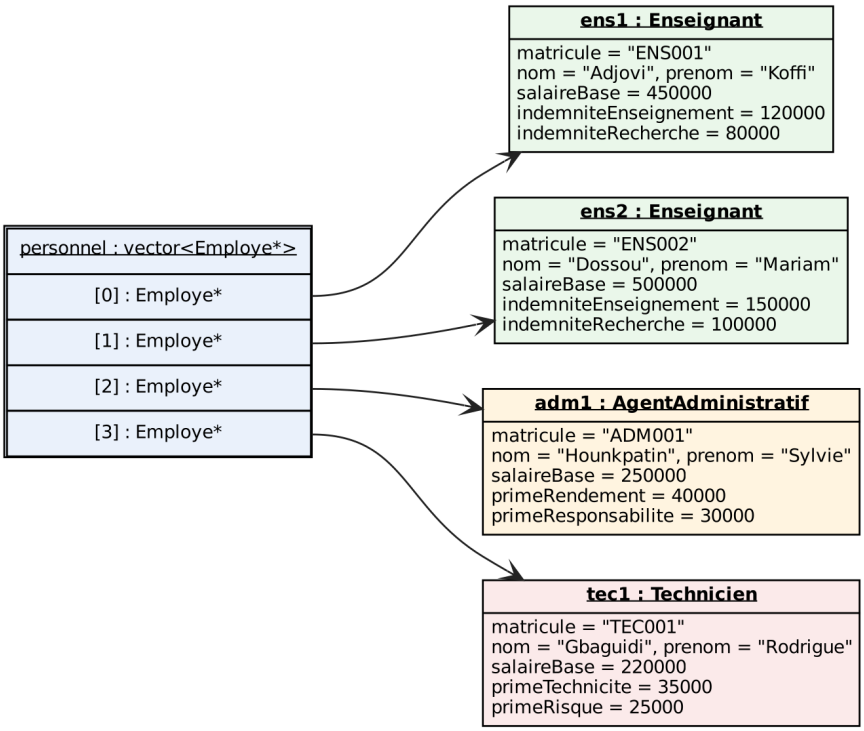



*Figure 3 – La collection : des pointeurs Employe* vers des objets de classes différentes*

## Résultat de l'exécution

```
--- Enseignant ---
Matricule       : ENS001
Nom et prenom   : Adjovi Koffi
Salaire de base : 450000 FCFA
Indemnite d'enseignement : 120000 FCFA
Indemnite de recherche   : 80000 FCFA
Salaire total   : 650000 FCFA
-----------------------------------
--- Enseignant ---
Matricule       : ENS002
Nom et prenom   : Dossou Mariam
Salaire de base : 500000 FCFA
Indemnite d'enseignement : 150000 FCFA
Indemnite de recherche   : 100000 FCFA
Salaire total   : 750000 FCFA
-----------------------------------
--- Agent administratif ---
Matricule       : ADM001
Nom et prenom   : Hounkpatin Sylvie
Salaire de base : 250000 FCFA
Prime de rendement       : 40000 FCFA
Prime de responsabilite  : 30000 FCFA
Salaire total   : 320000 FCFA
-----------------------------------
--- Technicien ---
Matricule       : TEC001
Nom et prenom   : Gbaguidi Rodrigue
Salaire de base : 220000 FCFA
Prime de technicite      : 35000 FCFA
Prime de risque          : 25000 FCFA
Salaire total   : 280000 FCFA
-----------------------------------
Masse salariale : 2000000 FCFA
```

**Vérification :** 650 000 + 750 000 + 320 000 + 280 000 = 2 000 000 FCFA, ce qui correspond à la masse salariale affichée. Si l'on écrit `Employe e(...);`, le compilateur refuse : « cannot declare variable to be of abstract type 'Employe' ».

## Notions mobilisées

| Notion | Où dans le code ? |
|---|---|
| **Héritage** | `class Enseignant : public Employe` |
| **Classe abstraite** | *Employe* contient une méthode virtuelle pure (`= 0`) |
| **Méthode virtuelle** | `virtual void afficher() const`, `virtual double calculerSalaire() const = 0` |
| **Redéfinition** | `calculerSalaire() const override` et `afficher() const override` dans chaque classe dérivée |
| **Polymorphisme** | Un même appel `e->calculerSalaire()` donne un résultat différent selon la catégorie |
| **Liaison dynamique** | Appel via un pointeur `Employe*` : la méthode est choisie à l'exécution |